Demystifying the Transformer Model

A Step-by-Step Mathematical Guide for Class 9 Students

Hello students! Today, we are going to dive into the magical world of Artificial Intelligence. You might have heard of ChatGPT or Google Gemini. But do you know the engine that powers them? It's called a **Transformer**.

Don't worry, we are not talking about the robots that turn into cars! In AI, a Transformer is a mathematical model that understands and generates human language. Let's break down the actual Python code from our notebook step-by-step, using simple Indian English and clear mathematics.

Our Running Example

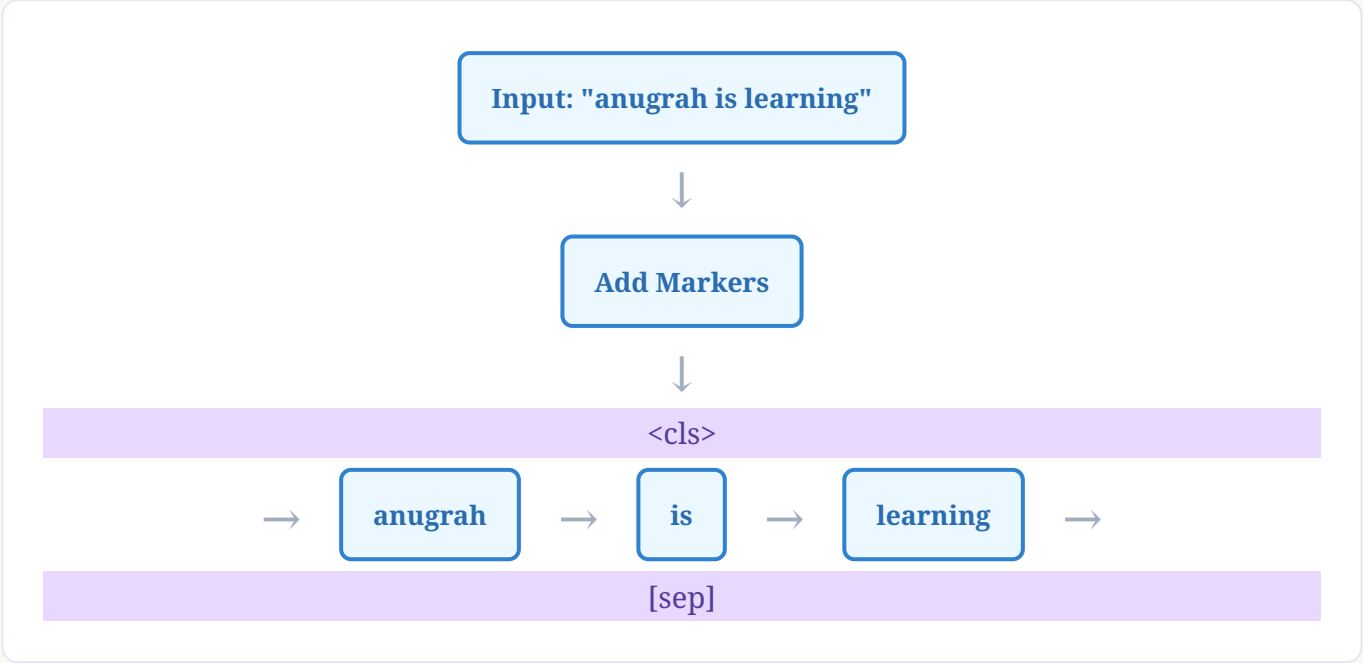
We will process a simple sentence: **"anugrah is learning"**. We will see exactly how the computer calculates numbers at every single step to understand this sentence!

Step 1: The Dictionary and Tokenization

Computers do not understand English letters. They only understand numbers. So, the first step is to give every word a unique Roll Number. This list of words and their roll numbers is called a **Vocabulary**.

```
Vocabulary Size: 39 words
Special Tokens:
<pad> = 0 (Padding - empty spaces)
<unk> = 1 (Unknown word)
<cls> = 2 (Start of sentence)
[sep] = 3 (Separator/End of sentence)
```

When we give the sentence *"anugrah is learning"* to the computer, it first adds start and end markers. Let's look at the diagram:



Mathematical Array Creation

We want all our inputs to be of a fixed length, say 10. If our sentence is short, we fill the empty seats with zeros (padding). According to our code output, the final converted numbers (Token IDs) look like this:

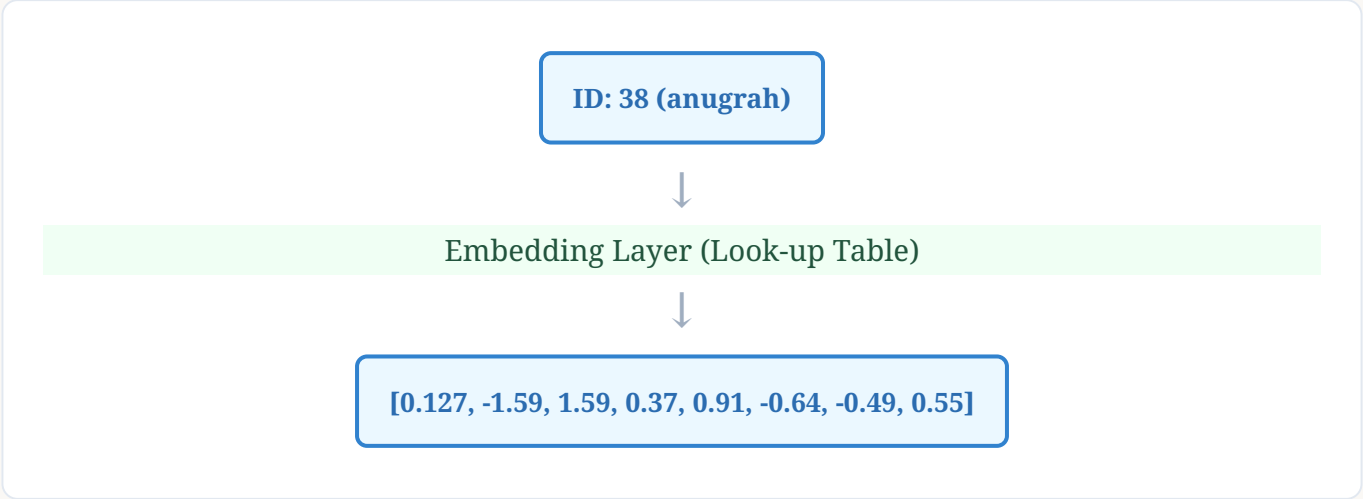
Input IDs = *[1, 38, 5, 9, 1, 0, 0, 0, 0, 0]*

(Fun Fact: You might wonder why the first and fifth numbers are 1 instead of 2 and 3. This is a tiny quirk in the provided code where the string markers failed to match the integer keys in the dictionary, so it gave them the "Unknown" ID of 1! Even programs make silly mistakes!)

Step 2: Word Embeddings (Giving Words a Meaning)

Just a single roll number is not enough to understand a word. Suppose I ask you to describe a student. You wouldn't just say "Roll No 38". You would give their marks in different subjects like Maths, Science, English, etc. Similarly, we convert each word ID into an array of decimal numbers. This is called an **Embedding**.

In our code, we set *d_model = 8*. This means every word is converted into an array of 8 numbers.



Shape Math: We had 1 sentence, with 10 words, and each word became 8 numbers. So the shape of our matrix is now $1 \times 10 \times 8$.

Step 3: Positional Encoding (The Sense of Order)

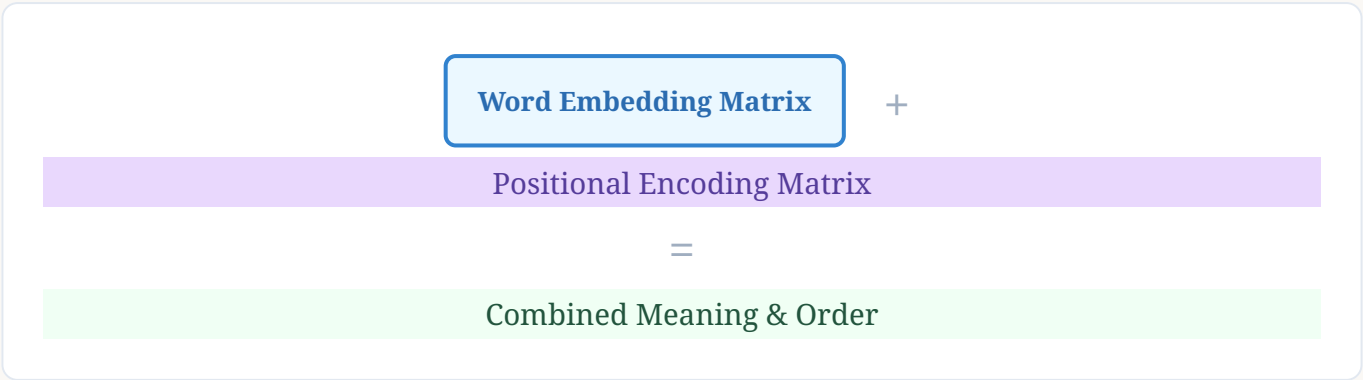
A sentence is not just a jumble of words. "Dog bites man" is very different from "Man bites dog". But the computer evaluates all words at the same time. To help the computer understand the order (first word, second word, etc.), we inject **Positional Encoding**.

We use mathematical Sine and Cosine waves to create a unique pattern for each position. The equations used in the code are:

For even positions: $$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

For odd positions: $$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

Here, pos is the word's position (0, 1, 2...) and i is the dimension index. We simply **add** these calculated wave numbers to our Word Embeddings.



Step 4: Segment Embeddings

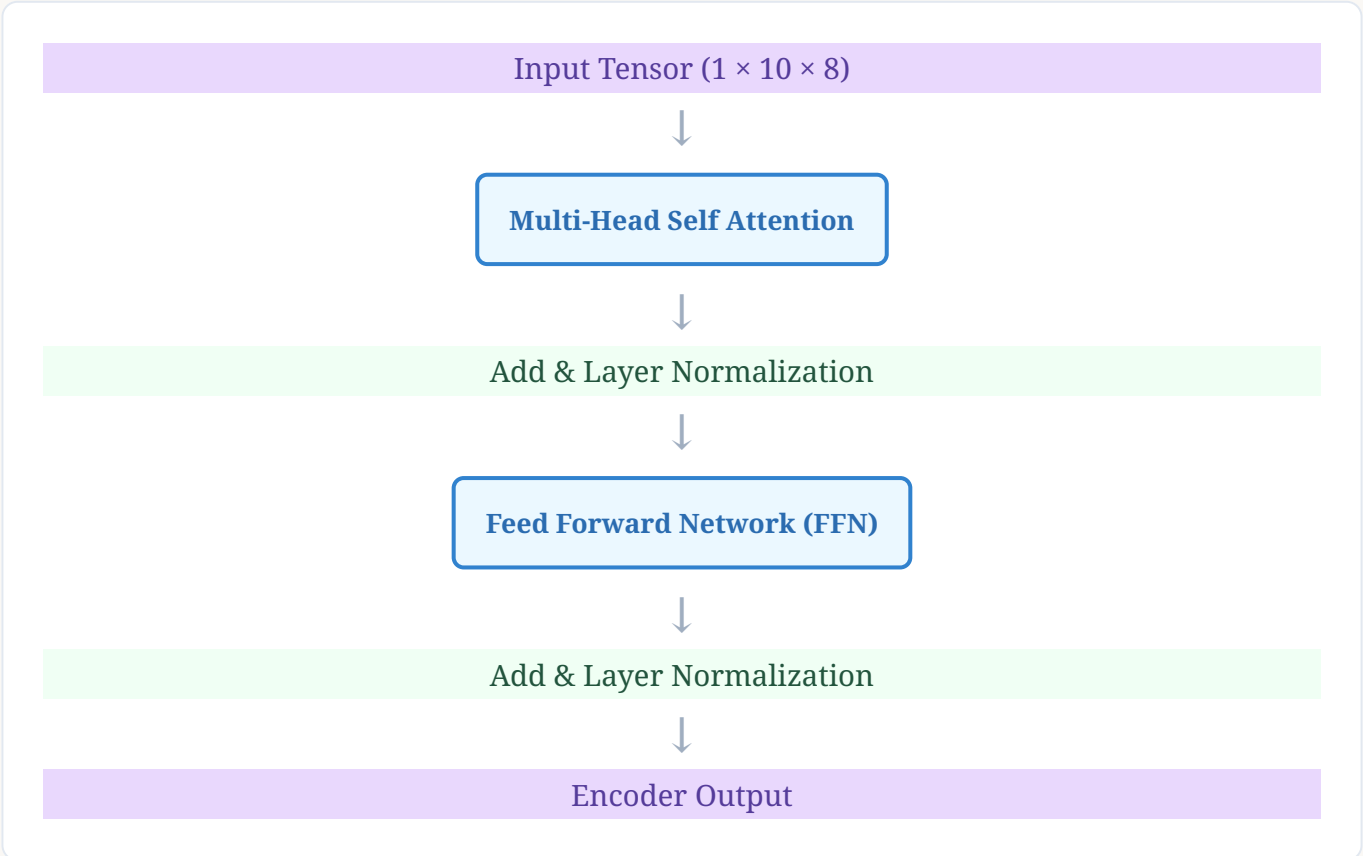
Sometimes, we feed two different sentences to the Transformer (like a Question and an Answer). Segment Embeddings simply tell the computer: "Hey, this word belongs to Sentence A, and that word belongs to Sentence B."

Since we only have one sentence ("anugrah is learning"), all words get a Segment ID of 0. These are also converted to 8-dimensional vectors and added to our combined matrix.

$$\textit{Final Input} = \textit{Word Embedding} + \textit{Positional Encoding} + \textit{Segment Embedding}$$

Step 5: The Encoder Layer (The Brain)

Now the real magic happens. The processed numbers enter the **Transformer Encoder**. Its job is to make words talk to each other to understand context. Let's look at its internal structure.



A. Multi-Head Self Attention (The Library Analogy)

Self-attention figures out which words are connected. When the model sees "anugrah is learning", it needs to know that "learning" is an action being done by "anugrah".

It creates three new vectors for each word using linear algebra (matrix multiplication):

- **Query (Q):** What I am looking for. (Like asking the librarian for AI books)
- **Key (K):** What I contain. (Like the title on the book's spine)
- **Value (V):** My actual meaning. (The content inside the book)

The Attention Equation:

$$Attention(Q, K, V) = softmax \left(rac{Q \cdot K^T}{\sqrt{d_k}} ight) \cdot V$$

Step-by-step math for Attention:

1. We multiply Q and K^T (dot product). This gives a "score" of how well two words match.
2. We divide by $\sqrt{d_k}$ (where d_k is the head dimension, $8/2 = 4$. So, $\sqrt{4} = 2$). This prevents the numbers from exploding and getting too big.
3. We apply a **Softmax** function. This converts the scores into percentages that sum up to 1 (like saying "learning" pays 80% attention to "anugrah" and 20% to "is").
4. Finally, we multiply by V to get the final context-aware word representation.

> **What is "Multi-Head"?**
>
> Instead of one person reading the sentence, imagine 2 students (Heads) reading it at the same time. One student looks for grammar, the other looks for emotions. In our code, $num\_heads = 2$. The 8 dimensions are split into 2 heads of 4 dimensions each! They do the math separately and then club their answers together.

B. Add & Norm (Preventing Forgetting)

After attention, the network uses a **Residual Connection**. It literally takes the original input and *adds* it to the attention output: $Output = Input + Attention(Input)$. This ensures the model doesn't forget the original word while learning the context.

Then, **Layer Normalization** is applied to keep the decimal values stable, making sure they don't fluctuate too wildly.

C. Feed Forward Network (The Evaluator)

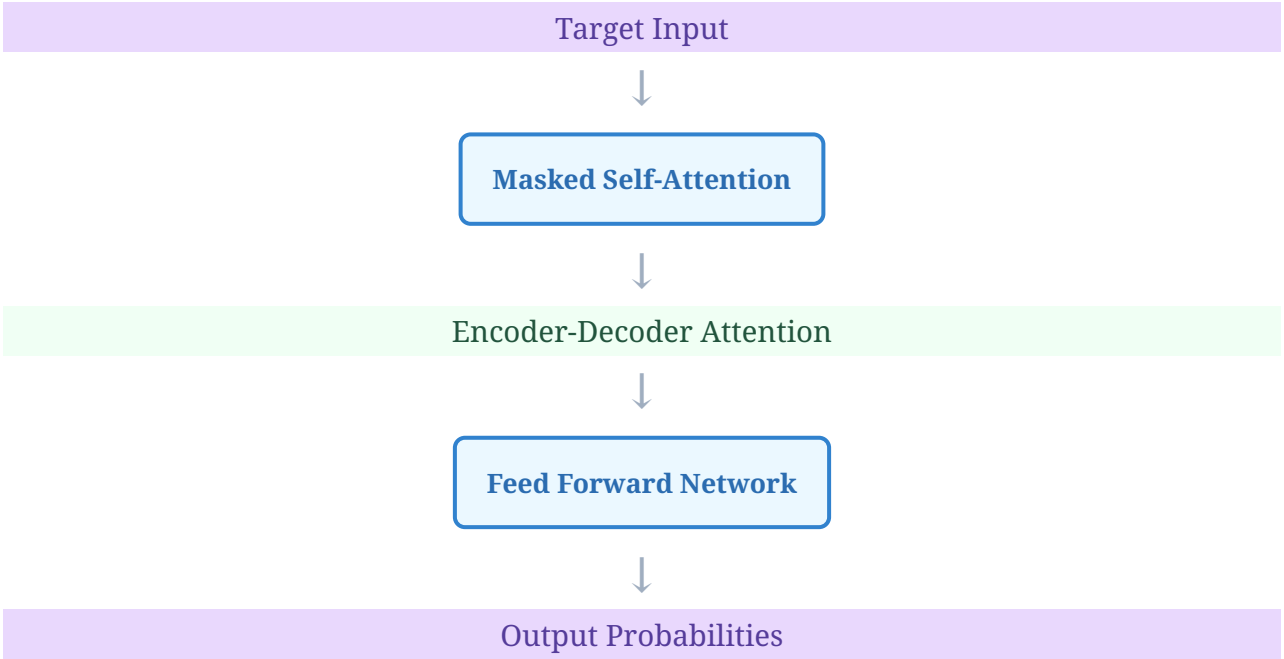
Every word vector is passed through a mini Neural Network (Linear → ReLU → Linear). In our code, the 8 dimensions are expanded to $ffn\_hidden\_dim = 32$ dimensions, and then shrunk back to 8.

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

The $\max(0, \dots)$ is the ReLU function. If a number is negative, it simply turns it into a zero. It filters out the useless information.

Step 6: The Decoder Layer (The Generator)

If we were translating English to Hindi, the Encoder reads the English sentence, and the **Decoder** generates the Hindi sentence word by word. The Decoder is very similar to the Encoder, but with two big differences:



- **Masked Self-Attention:** When the Decoder predicts the 3rd word, it is *not allowed* to look at the 4th word. We apply a "mask" (turning future numbers into negative infinity, $-\infty$). When passed through Softmax, $e^{-\infty} = 0$, meaning 0% attention is given to the future. It's like hiding the answers in a textbook so the student doesn't cheat!
- **Encoder-Decoder Attention:** Here, the Query (Q) comes from the Decoder (what Hindi word should I write next?), but the Key (K) and Value (V) come directly from the **Encoder's output** (the English context).

Step 7: The Final Output (Prediction)

Finally, the 8-dimensional output from the Decoder goes into a Final Linear Layer. This layer stretches the 8 numbers back into 39 numbers (our vocabulary size).

$$\textit{Linear(8 dimensions) ightarrow 39 dimensions}$$

We use Softmax one last time to turn these 39 numbers into probabilities. The word with the highest probability (say, 95% for the word "learning") becomes the model's final predicted output!

Summary

By tracking **"anugrah is learning"**, we saw how words turn into IDs, how they gain multi-dimensional meaning, how sine waves give them order, and how Q, K, V math helps them understand each other. Lakhs of these calculations happen in a fraction of a second to make AI systems work. Now you know the secret math behind the magic!